OpenVLA, From Pixels to Motor Commands

A study guide to the architecture, training, and inference of OpenVLA-7B, written for interpretability work

Prepared for the VLAinterp project

October 2026

Abstract

OpenVLA [1] is a 7.2B-parameter vision-language-action model. It reuses a vision-language model, Prismatic-7B [2], almost unchanged. The model reads one RGB image and one language instruction and writes a robot action as seven ordinary tokens from the language model's vocabulary. This guide sorts what you need to know into six parts: (I) the one-paragraph mental model and where OpenVLA came from; (II) the forward pass, one component at a time, with exact tensor shapes; (III) action tokenization and de-tokenization; (IV) pretraining data, objective, and the design choices the authors tested; (V) fine-tuning, deployment, and results, including the LIBERO recipe used in this project; (VI) the consequences for interpretability, plus a short spoken explanation and self-test questions. Numbers come from the OpenVLA paper, the released Hugging Face code and config, and related papers. Section 22 lists the sources.

Contents

I The Big Picture **4**

1 OpenVLA in one paragraph **4**

2 Where OpenVLA comes from **5**

II The Forward Pass, Component by Component **5**

3 Inputs **5**

3.1 Image preprocessing 5

3.2 Prompt 6

4 Vision encoders: DINOv2 + SigLIP **6**

5 The projector **7**

6 Assembling the sequence **7**

7 The language model: Llama 2 7B **8**

III Actions as Tokens **8**

8 From a real action to a token **9**
8.1 Action space 9
8.2 Step 1: normalize to $[-1, 1]$ per dataset 9
8.3 Step 2: discretize into bins 9
8.4 Step 3: write the bin as a vocabulary token 9

9 From tokens back to an action **9**

IV Training **11**

10 Data: Open X-Embodiment, curated **11**

11 Objective **11**

12 Recipe and compute **12**

13 Design choices the authors tested **12**

V Adaptation, Deployment, and Results **12**

14 Fine-tuning to a new robot or task **12**
14.1 The LIBERO recipe (what your checkpoints are) 13

15 Inference **13**

16 Headline results **14**

17 Limitations and what came next **14**

VI Mental Model for Interpretability **15**

18 Tensor-shape cheat sheet (batch 1) **15**

19 Where to look, and what each probe means **15**

20 The two-minute spoken explanation **16**

21 Self-test questions **16**

22 Sources **17**

Part I
The Big Picture

1 OpenVLA in one paragraph

Mental model: the whole model

OpenVLA is a chatbot that answers in robot actions. Take a standard image-to-text model ("describe this picture"). Ask it, in a fixed template, *"What action should the robot take to {instruction}?"* Train it to answer with 7 tokens. Each token stands for one of 256 bins of one action dimension: $\Delta x, \Delta y, \Delta z$, Δroll, Δpitch, Δyaw, gripper. The 7 tokens are converted back to numbers and sent to the robot. A new camera image arrives and the model is asked again. Nothing about the transformer is robot-specific. All of the robotics lives in (a) what the 7 output tokens *mean*, and (b) the 970k robot demonstrations the model was trained on.

The pipeline has four stages. Every later section expands one of them:

1. **See.** Two pretrained vision transformers, DINOv2 and SigLIP, each turn the 224×224 image into 256 patch features. The two feature vectors for each patch are concatenated.
2. **Translate.** A small MLP, the *projector*, maps each of the 256 patch features into the word-embedding space of Llama 2. After this, the image is just 256 extra "words".
3. **Think.** Llama 2 7B, a 32-layer causal transformer, processes [<s>] + 256 image tokens + the instruction prompt.
4. **Act.** The language-model head predicts 7 tokens one at a time. Each token is looked up in a bin table and rescaled with per-dataset statistics into a real action.

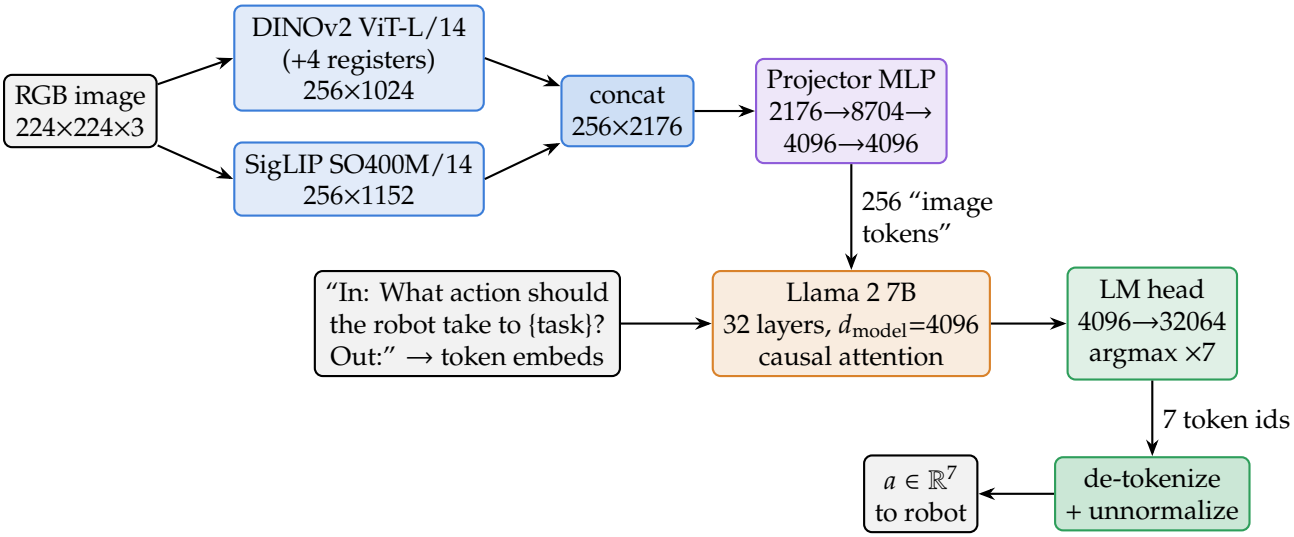


Figure 1: OpenVLA-7B end to end. Colors: blue = vision, purple = projector, orange = language model, green = action read-out. All parameters, including both vision encoders, are trained during VLA training.

2 Where OpenVLA comes from

A VLA is easier to explain once you know the ideas it borrows.

RT-1 (Google, 2022)
A robot transformer trained from scratch on about 130k demonstrations. It introduced **256-bin discretization of each action dimension**, with actions predicted as classification targets. OpenVLA keeps this scheme.

RT-2 (Google DeepMind, 2023)
Fine-tuned large VLMs (PaLI-X 55B, PaLM-E 12B) to emit action bins *as text tokens*, co-trained with web vision-language data. This is the original "actions are just language" VLA. RT-2-X is RT-2 trained on Open X-Embodiment. It was the main closed-source baseline OpenVLA beat.

Open X-Embodiment (2023)
A pooled dataset of 1M+ real-robot trajectories from 22 robot embodiments, in a common RLDS format. OpenVLA trains on a curated 970k-trajectory subset.

Octo (2024)
An open 93M-parameter generalist policy with a *diffusion* action head. Its data-mixture weights were the starting point for OpenVLA's.

Prismatic VLMs (2024)
A controlled study of VLM design choices [2]. It produced the backbone OpenVLA starts from: fused DINOv2+SigLIP vision, a Llama 2 base model, naive resize, single-stage training.

OpenVLA (June 2024)
Kim, Pertsch, Karamcheti, et al. (Stanford, UC Berkeley, TRI, Google DeepMind, Physical Intelligence, MIT). Prismatic-7B + RT-2-style action tokens + OpenX data, fully open (weights, code, data pipeline).

Successors
OpenVLA-OFT (2025) [3] replaces the token output with parallel, continuous, chunked actions (Section 17). π_0 uses a flow-matching "action expert". FAST compresses actions with a DCT before tokenizing. These are mostly reactions to the bottlenecks of OpenVLA's one-token-per-dimension design.

Part II
The Forward Pass, Component by Component

3 Inputs

3.1 Image preprocessing

- **One image, one camera, one time step.** OpenVLA takes a single third-person RGB frame. It has no wrist camera, no proprioception (joint angles or gripper width), and no history of past frames or actions. The released `norm_stats` even store `proprio` statistics as all zeros.

- **Resize.** The image is resized directly to 224×224 (`image_resize_strategy = "resize-naive"`), without cropping or letterboxing. The authors also tried 384 × 384. It gave no measurable improvement and trained about 3× slower.
- **Two normalizations, one tensor.** Each encoder expects different pixel statistics: ImageNet mean/std for DINOv2, mean = std = 0.5 for SigLIP. The processor normalizes the same image twice and stacks the results along the channel axis, so `pixel_values` has shape $[B, 6, 224, 224]$. The model splits it back into two 3-channel images.
- **Augmentation.** During training, each image gets a random crop covering 90% of its area, resized back to 224. At test time you therefore take a *center* 90% crop (`--center_crop True` in the LIBERO scripts). Skipping this measurably lowers success.

3.2 Prompt

The text input is always the same template, with the task instruction filled in:

```
In: What action should the robot take to {instruction}?
Out:
```

The Llama SentencePiece tokenizer adds a <s> (BOS, id 1) at the start. At training time the target text `"Out: <7 action tokens></s>"` tokenizes the space after "`Out:`" as a separate token, id **29871** ("_"). At inference, `predict_action` therefore appends token 29871 if the prompt doesn't already end with it, so the model sees exactly the prefix it was trained on. The first action token is predicted at the position of token 29871.

4 Vision encoders: DINOv2 + SigLIP

	DINOv2	SigLIP
timm id	`vit_large_patch14_reg4_dinov2.lvd142m`	`vit_so400m_patch14_siglip_224`
Architecture	ViT-L/14, 24 blocks, width 1024	ViT-SO400M/14, 27 blocks, width 1152
Pretraining	Self-supervised (image-only), LVD-142M	Contrastive image–text (sigmoid loss)
What it is good at	Geometry, spatial layout, part structure	Semantics, object categories, language alignment
Extra tokens	CLS + 4 *register* tokens (discarded)	none (no CLS)
Output used	patch tokens of the 2nd-to-last block	patch tokens of the 2nd-to-last block
Output shape	256 × 1024	256 × 1152

Patch grid. With patch size 14 at 224 × 224, each encoder sees a 16 × 16 grid, so **256 patches**. Each patch covers 14 × 14 pixels. Patch i in DINOv2 and patch i in SigLIP cover the same pixels, so their features can be concatenated per patch: 1024 + 1152 = **2176** dimensions per patch. The code is literally `torch.cat([patches, patches_fused], dim=2)`.

Why the penultimate layer? Both featurizers call `get_intermediate_layers(n={len(blocks)-2})`. The last block of a pretrained ViT is specialized for its pretraining objective (e.g. the contrastive head). The second-to-last block is a more general-purpose representation. This convention comes from LLaVA.

Why two encoders? Prismatic [2] found that contrastive encoders (CLIP, SigLIP) beat DINOv2 alone on VLM benchmarks. *Adding* DINOv2 features to SigLIP still gave significant gains, 5–10% on localization-heavy tasks. For robots, spatial precision matters even more. OpenVLA's ablation trained on Bridge only: dropping DINOv2 (SigLIP alone) cost about 5 points of success (45.6% → 40.6%).

Mental model: the two encoders

SigLIP tells the model *what* things are ("a black bowl, a plate"). DINOv2 tells it *where* things are and how they're shaped. Concatenation means every image token carries both kinds of description of one 14×14 pixel square.

5 The projector

The fused-backbone projector is a 3-layer MLP applied independently to each patch:

$$z_i = W_3\,\mathrm{GELU}\big(W_2\,\mathrm{GELU}(W_1 v_i + b_1) + b_2\big) + b_3, \qquad v_i \in \mathbb{R}^{2176},\ z_i \in \mathbb{R}^{4096} \tag{1}$$

with $W_1 : 2176 \to 8704$ (4× expansion), $W_2 : 8704 \to 4096$, $W_3 : 4096 \to 4096$. That is about 71M parameters. The paper calls it a "2-layer MLP". The released fused variant has three linear layers, as above. Its only job is to put visual features into the same vector space as Llama's word embeddings, so the language model can treat them like words. It does not mix information between patches: there is no attention and no pooling, and 256 vectors go in and 256 come out.

6 Assembling the sequence

The projected patches are spliced in *right after BOS*:

```
# Prismatic defaults to inserting after <BOS> token (1:)
multimodal_embeddings = torch.cat(
    [input_embeddings[:, :1, :], projected_patch_embeddings, input_embeddings[:, 1:, :]],
        dim=1)
```

Patch positions get attention mask `True` and label `-100`, so the loss ignores them.

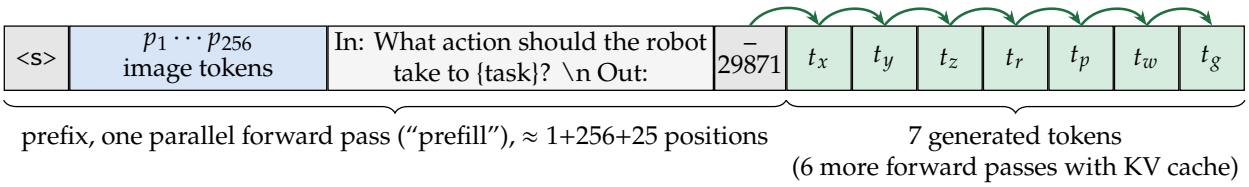


Figure 2: The token sequence. Green arrows: the residual stream at the *tail* of each arrow produces the logits for the token at its *head*. So t_x is read out at the "_" position, t_y at the t_x position, and so on. During training the 7 ground-truth action tokens and the closing </s> are supervised. Everything else is masked out of the loss.

Interpretability note: the causal mask covers the image too

Llama uses a strictly causal mask, and the image comes *before* the text. Two consequences:

1. **Image tokens never see the instruction.** Inside the LLM, the representations at patch positions are computed without the task text. Any instruction-dependent processing of the scene has to happen at text positions or action positions, which can attend back to the patches.
2. **Image tokens are causally ordered in raster order.** Patch 17 can attend to patches 1–16 but not 18–256. The vision encoders already mixed information globally, so this matters less than it sounds. Still, patch positions are not symmetric inside Llama.

7 The language model: Llama 2 7B

Layers	32 decoder blocks
Residual width d_{model}	4096
Attention	32 heads × 128 dims, multi-head (no GQA at 7B), RoPE positions
MLP	SwiGLU, hidden width 11008
Normalization	RMSNorm, pre-norm (before attention and before MLP), plus a final RMSNorm
Vocabulary	32000 real tokens, padded to 32064 (multiple of 64); pad id 32000
Embeddings	*untied*: input embedding matrix W_E and output head W_U are separate
Parameters	≈6.7B (of OpenVLA's ≈7.2B total)

Each block, in residual-stream form (x_ℓ is the stream entering layer ℓ):

$$h_\ell = x_\ell + \text{Attn}_\ell\big(\text{RMSNorm}(x_\ell)\big) \tag{2}$$

$$x_{\ell+1} = h_\ell + W^\ell_{\text{down}}\Big(\text{SiLU}\big(W^\ell_{\text{gate}}\,\tilde{h}_\ell\big) \odot W^\ell_{\text{up}}\,\tilde{h}_\ell\Big), \qquad \tilde{h}_\ell = \text{RMSNorm}(h_\ell) \tag{3}$$

and the output is

$$\text{logits} = W_U\,\text{RMSNorm}_{\text{final}}(x_{32}) \in \mathbb{R}^{32064}. \tag{4}$$

Base, not chat. OpenVLA uses `meta-llama/Llama-2-7b-hf` (the pure base model, `llama2-7b-pure`). Prismatic found instruction-tuned LMs gave no benefit and hallucinated more.

Mental model: the residual stream

Picture each of the ~290 positions as a 4096-dimensional "notepad". The notepad at a patch position starts with a description of one image square. The notepad at a text position starts with a word embedding. Each of the 32 layers *adds* to the notepads. Attention copies information between positions, always from earlier to later. The MLP rewrites information within a position. At the end, the notepad at the "_" position is multiplied by W_U to score all 32064 tokens, and the top score among the action tokens is the bin for Δx.

Part III
Actions as Tokens

8 From a real action to a token

8.1 Action space

The training data are mostly 7-DoF *end-effector delta* actions: a change in position $(\Delta x, \Delta y, \Delta z)$, a change in orientation (Δroll, Δpitch, Δyaw), and a gripper command. Datasets are only kept if they have single-arm end-effector control and a third-person camera. The units and scale of the action differ by dataset. That is why normalization is done per dataset.

8.2 Step 1: normalize to $[-1, 1]$ per dataset

For each dataset D and each dimension j, compute the 1st and 99th percentiles $q^D_{01,j}, q^D_{99,j}$ of the training actions. Map that interval to $[-1, 1]$ and clip:

$$\bar{a}_j = \text{clip}\left(2\,\frac{a_j - q_{01,j}}{q_{99,j} - q_{01,j}} - 1,\ -1,\ 1\right). \tag{5}$$

Why quantiles instead of min/max? One outlier action in a dataset would stretch a min/max range and squeeze all normal actions into a few bins. With quantiles the bins cover where the data actually are. Only the most extreme 2% of values get clipped. In Bridge, for example, Δx has min/max $\approx \pm 0.41$ but $q_{01}/q_{99} \approx \pm 0.028$. A min/max grid would be 15× coarser.

The gripper is not normalized. It has `mask = False` in the stats and is already in $[0, 1]$ (0 = close, 1 = open in the OpenVLA training convention).

8.3 Step 2: discretize into bins

```
bins = np.linspace(-1, 1, 256) # 256 bin EDGES
bin_centers = (bins[:-1] + bins[1:]) / 2.0 # 255 centers
idx = np.digitize(np.clip(a, -1, 1), bins) # integer in 1..256
token_id = tokenizer.vocab_size - idx # 32000 - idx -> 31744..31999
```

8.4 Step 3: write the bin as a vocabulary token

Llama's tokenizer has only 100 reserved special tokens, too few for 256 bins. OpenVLA instead **overwrites the last 256 tokens of the Llama vocabulary** (ids 31744–31999). The paper describes these as the least-used tokens. They are rare sub-word pieces, and their old meaning is destroyed by training. Their input embeddings and their rows of W_U are then learned as action bins.

9 From tokens back to an action

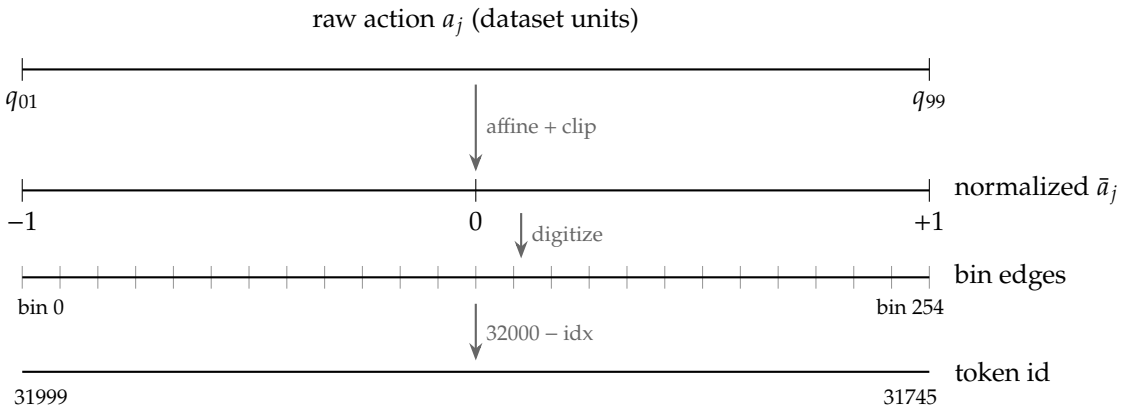


Figure 3: The tokenization chain for one action dimension. Note the **reversal**: *larger* token ids mean *more negative* actions.

```
generated_ids = self.generate(input_ids, max_new_tokens=7, do_sample=False)
ids = generated_ids[0, -7:]
idx = np.clip(32000 - ids - 1, 0, 254) # bin index into 255 centers
norm = bin_centers[idx] # in [-1, 1]
a = np.where(mask, 0.5*(norm+1)*(q99-q01) + q01, norm) # unnormalize
```

- **Decoding is greedy** (`do_sample=False`): argmax at each of 7 steps, each conditioned on the previous action tokens.
- **The argmax runs over all 32064 logits**, not only the 256 action rows. A trained model essentially always picks an action token. If it didn't, the clip would silently turn the stray token into an edge bin.
- **Which statistics?** `unnorm_key` picks the dataset whose q_{01}/q_{99} are used, e.g. `"bridge_orig"` for the base model or `"libero_spatial"` for the LIBERO checkpoint. The *same* token means a different physical motion under different keys.
- **Resolution.** One bin is 2/255 of the normalized range, or $(q_{99} - q_{01})/255$ in raw units. In your run 2 data that is about 0.006–0.007 for translation.

Common misconception: "256 bins"

The paper says 256 bins, and 256 token ids are reserved. The code builds 256 bin *edges*, so there are only 255 *centers*. On decode, ids 31744 and 31745 both map to the last center (+0.996). In practice the model has 255 distinct output values per dimension, and the lowest token id is effectively a duplicate. When your logit lens takes a softmax over "256 action bins", keep in mind that two of those rows decode to the same value.

Common misconception: the 7 dimensions are independent outputs

They are *not* independent. Generation is autoregressive in the fixed order x, y, z, roll, pitch, yaw, gripper. The gripper token is predicted with the first six already in context. During training this was teacher-forced on the *ground-truth* previous dimensions.

At test time the model conditions on its *own* previous predictions.

Part IV
Training

10 Data: Open X-Embodiment, curated

- **Source.** Open X-Embodiment (OpenX): 70+ datasets, 2M+ trajectories when the paper was written.
- **Curation.** Keep only manipulation datasets with ≥ 1 third-person camera and single-arm end-effector control. Result: **970k trajectories** across about 25 datasets (the 25 `norm_stats` keys in the released config).
- **Mixture weights.** Start from Octo's weights, which down-weight low-diversity datasets and up-weight diverse ones. The largest shares are Bridge 13.3%, Fractal (RT-1) 12.7%, and Kuka 12.7%. DROID was added at 10% but removed for the last third of training because its action-token accuracy stayed low.
- **Cleaning.** All-zero (no-op) actions in Bridge were filtered out.
- **No web data.** Unlike RT-2, OpenVLA does *not* co-train with internet vision-language data during robot training. Its "world knowledge" comes only from the pretrained Prismatic weights it starts from.

11 Objective

Standard next-token cross-entropy, computed **only on the action tokens** (and the closing </s>):

$$\mathcal{L} = -\sum_{j=1}^{7} \log p_\theta\big(t_j \mid \text{image},\ \text{prompt},\ t_{<j}\big). \tag{6}$$

Note what this loss does *not* know: bin 128 and bin 129 are as "different" as bin 0 and bin 255. Getting close earns nothing. Any notion that neighboring bins are similar has to be learned from the data. This is one reason successors switched to regression (Section 17).

The training metric reported in the paper is **action-token accuracy**: the fraction of action tokens whose argmax equals the ground-truth bin, under teacher forcing.

12 Recipe and compute

Initialization	Prismatic-7B (DINOv2+SigLIP, Llama 2 7B), already a VLM
Trainable	**everything**, including both vision encoders
Hardware / time	64 A100 GPUs × 14 days ≈ 21,500 A100-hours
Batch size	2048
Learning rate	2×10^{-5}, constant, no warmup (warmup did not help)
Epochs	27 passes over the mix, until training action-token accuracy > 95%
Stack	PyTorch, FSDP, FlashAttention, AMP (bf16)

Common misconception: "freeze the vision encoder, like VLMs do"

Prismatic found that fine-tuning the vision encoder *hurt* VLM benchmarks. OpenVLA found the opposite for control: fine-tuning it was crucial. Averaged across their evaluations, fine-tuning reached about 80% success versus about 47% with a frozen encoder. The likely reason is that internet-pretrained features are not precise enough about small spatial details (exact gripper–object offsets) for control. 27 epochs is also far more than the usual 1–2 for VLMs. Robot policies need near-exact action-token accuracy.

13 Design choices the authors tested

Choice	Finding
VLM backbone	LLaVA beat IDEFICS-1 by about 35% absolute on language-grounding tasks. Prismatic beat LLaVA by about 10%, which the authors credit to the fused SigLIP+DINOv2 encoder.
OpenX vs. Bridge-only data	Training only on Bridge cost about 30 points on Bridge tasks (76.3% → 45.6%). Cross-embodiment data helps even on the in-domain robot.
DINOv2	Removing it (SigLIP only, Bridge-only training) cost about 5 more points (45.6% → 40.6%).
Resolution	384 px vs. 224 px: no gain, about 3× the training compute.
Frozen vision	Large drop (above).
Epochs	Many more than for VLMs. Success kept improving until token accuracy was above 95%.

Part V
Adaptation, Deployment, and Results

14 Fine-tuning to a new robot or task

The authors compared fine-tuning strategies on Franka tasks (batch 16):

Strategy	Success	Trainable params	VRAM
Full fine-tuning	69.7 ± 7.2%	7,188.1M	163.3 GB
Last layer only	30.3 ± 6.1%	465.1M	51.4 GB
Frozen vision	47.0 ± 6.9%	6,760.4M	156.2 GB
Sandwich (vision + embed + last layer)	62.1 ± 7.9%	914.2M	64.0 GB
LoRA, $r = 32$	**68.2 ± 7.5%**	**97.6M (1.4%)**	59.7 GB
LoRA, $r = 64$	68.2 ± 7.8%	195.2M	60.5 GB

LoRA ($r = 32$, applied to all linear layers) matches full fine-tuning at 1.4% of the trainable parameters. It runs in 10–15 hours on one A100, about 8× less compute. All released LIBERO checkpoints were trained this way.

14.1 The LIBERO recipe (what your checkpoints are)

1. **Re-render demos at** 256×256, up from LIBERO's native 128×128.
2. **Rotate images 180°**, at train and test time, because LIBERO's renderer returns them upside down.
3. **Filter no-op actions**: near-zero translation and rotation with no gripper change. The authors call this crucial. Without it the policy learns to stand still and freezes during evaluation.
4. **Drop failed demos** found by replaying them: 68/500 for Spatial, 46 for Object, 72 for Goal, 121 for Long.
5. **Third-person camera only.** No wrist camera and no proprioception, for every method compared.
6. One LoRA-r=32 checkpoint per suite, e.g. `openvla/openvla-7b-finetuned-libero-spatial`, with its own `unnorm_key` (`libero_spatial`).
7. **Evaluation-time conventions**: center crop (90% area), then convert the gripper. OpenVLA outputs $[0, 1]$. LIBERO expects -1 = open and $+1$ = close, so the value is rescaled to $[-1, 1]$, binarized, and its sign flipped.

15 Inference

- **Cost per control step:** one prefill pass over about 280 positions, then 6 more single-token passes with the KV cache. That is 7 sequential LM-head read-outs per action.
- **Speed and memory:** about 15 GB in bf16, about 6 Hz on an RTX 4090 without compilation or speculative decoding.
- **Quantization** (8 Bridge tasks):

Precision	Success	VRAM
bf16	71.3 ± 4.8%	16.8 GB
int8	58.1 ± 5.1%	10.2 GB
int4	71.9 ± 4.7%	7.0 GB

int8 looks worse only because its kernels were slow (1.2 Hz on an A5000), and the real robot kept moving while the model computed. With blocking control, where the robot waits for each action, the three precisions perform the same (70.0 / 74.4 / 68.8%).

16 Headline results

- **Zero-shot generalist control** (BridgeData V2 WidowX, 17 tasks; Google Robot, 12 tasks): OpenVLA beats RT-2-X (55B) by **16.5% absolute** averaged over 29 tasks, with 7× fewer parameters. It outperforms RT-2-X in every category except *semantic* generalization, where RT-2-X's web co-training helps.
- **Fine-tuning to new setups** (Franka): about 20% better than Diffusion Policy trained from scratch, and the only method with ≥ 50% success on every task. Diffusion Policy is still smoother on narrow, dexterous single-instruction tasks.
- **LIBERO** (500 trials per suite, 3 seeds):

Method	Spatial	Object	Goal	Long	Avg
Diffusion Policy (scratch)	78.3	**92.5**	68.3	50.5	72.4
Octo (fine-tuned)	78.9	85.7	**84.6**	51.1	75.1
OpenVLA (LoRA)	**84.7**	88.4	79.2	**53.7**	**76.5**
OpenVLA-OFT		(reported average)			97.1

Your run 2 (42/50 = 84.0% on Spatial) is consistent with the reported 84.7%.

17 Limitations and what came next

Limitations stated by the authors. Single image only: no history, no proprioception, no wrist camera. Throughput is too low for high-frequency control such as ALOHA at 50 Hz. Reliability is usually below 90%. Open questions remain about base-VLM size and whether web co-training helps.

Structural bottlenecks of the token design. (1) *Sequential decoding*: 7 forward passes per action. (2) *Discretization*: 255 values per dimension, and a loss that ignores how close a wrong bin is. (3) *One step at a time*: no action chunking, so motion can be jerky and small errors compound.

OpenVLA-OFT (Kim, Finn, Liang, 2025) [3] keeps the same backbone and changes the action interface:

- **Parallel decoding**: feed empty action placeholder embeddings and use *bidirectional* attention over them, so all action dimensions come out in one forward pass.
- **Action chunking**: predict K future actions at once ($K = 8$ on LIBERO).
- **Continuous actions**: an MLP head on the final hidden states replaces the LM head and bins.

- **L1 regression loss** in place of cross-entropy. It matches diffusion heads on success while training and running faster.
- Optional extra inputs (wrist camera, proprioception). The OFT+ variant adds FiLM language conditioning for bimanual ALOHA.

Result: LIBERO average 76.5% → 97.1%, and about 26× action-generation throughput.

Part VI
Mental Model for Interpretability

18 Tensor-shape cheat sheet (batch 1)

Stage	Shape	Notes
Env frame (LIBERO)	$256 \times 256 \times 3$	rotate 180°, center crop 90%, resize
`pixel_values`	$[1, 6, 224, 224]$	DINOv2-normed ‖ SigLIP-normed
DINOv2 patches	$[1, 256, 1024]$	block 22 of 24 (0-indexed); CLS + registers dropped
SigLIP patches	$[1, 256, 1152]$	block 25 of 27 (0-indexed)
Fused features	$[1, 256, 2176]$	concat on feature dim
Projected	$[1, 256, 4096]$	3-layer GELU MLP
LLM input	$[1,\ 1{+}256{+}T,\ 4096]$	$T \approx 20$–35 prompt tokens incl. 29871
Residual stream x_ℓ	same	$\ell = 0$ (embeddings) … 32
Logits	$[1, L, 32064]$	action rows: ids 31744–31999
Output	$\mathbb{R}^7$	after de-tokenize + unnormalize + gripper convention

19 Where to look, and what each probe means

Interpretability note: reading action predictions from inside

- **Which position?** The prediction for action dimension j (1-indexed) is read from position $P_{\text{end}} + j - 1$, where P_{end} is the "_" (29871) position. Dimension x is read at a *text* position. Dimensions y through gripper are read at *action-token* positions, whose input embeddings are learned bin embeddings.
- **Logit lens on actions.** $\text{logit-lens}_\ell = W_U[\,31744{:}32000\,] \cdot \text{RMSNorm}_{\text{final}}(x_\ell)$. You need the final RMSNorm, because the stream's scale grows with depth. Restricting to the 256 action rows (your run 2 setup) answers "which bin would the model pick now?" and ignores whether a text token would have won.
- **Untied embeddings.** When you analyze action tokens as *inputs* (e.g. the x-token feeding the y prediction), the relevant vector is a row of W_E, not W_U. These are different learned vectors.
- **Bin geometry.** Cross-entropy does not force neighboring bins to have similar W_U rows.

Whether they end up ordered (e.g. lying near a 1-D curve) is an empirical question, and a cheap first analysis.

- **Markov policy.** With no history in the input, any temporal consistency you see across steps (e.g. a gripper that stays closed) has to be recomputed from the current image each step.
- **Image tokens are instruction-blind** (causal mask). Probing patch positions for task identity should fail by construction. Probing text or action positions for scene properties should succeed.
- **FFN value vectors.** Häon et al. (CoRL 2025) [4] project rows of W_{down} through the unembedding into token space. They find sparse, semantically meaningful directions in OpenVLA and π_0 (e.g. speed, direction) and steer behavior by injecting them at inference time. This is the natural causal follow-up to logit-lens observations.

20 The two-minute spoken explanation

"OpenVLA is a vision-language model repurposed as a robot policy. It starts from Prismatic, a LLaVA-style VLM. Two vision transformers, DINOv2 for spatial structure and SigLIP for semantics, each turn a 224×224 camera image into a 16×16 grid of patch features. Those are concatenated and passed through a small MLP into Llama 2's embedding space. That gives 256 visual tokens, placed right after the start token and before a fixed prompt: 'What action should the robot take to *[instruction]*?' Llama 2 7B processes the sequence causally, and the language-model head generates seven tokens. Each one is a bin index for one action dimension: three position deltas, three rotation deltas, and the gripper. To fit them in the vocabulary, the authors overwrote the 256 least-used Llama tokens. Each action dimension is normalized per dataset with its 1st and 99th percentiles and cut into uniform bins, and at test time that mapping is inverted. The whole 7B model, vision encoders included, was trained with plain cross-entropy on 970k Open X-Embodiment trajectories for 27 epochs on 64 A100s. It beats the 55B RT-2-X by 16.5 points. It adapts to new robots with LoRA at 1.4% of the parameters and gets about 77% on LIBERO. Its main limits come from the token interface: seven sequential decoding steps, discretized actions, no history, and no chunking. OpenVLA-OFT fixes these with parallel decoding of continuous action chunks trained with an L1 loss."

21 Self-test questions

1. Why does OpenVLA concatenate DINOv2 and SigLIP features instead of using one encoder? What did the ablation cost when DINOv2 was removed?
2. Where exactly are image tokens inserted in the sequence? Can a patch token attend to the instruction?
3. Write the formula mapping a raw action value to a token id. Why use q_{01}/q_{99} instead of min/max?

4. How many distinct values can one action dimension take at output? Why is the answer not 256?
5. At which sequence position is the gripper token predicted? What has the model already written by then?
6. What does `unnorm_key` do, and why does it matter?
7. VLMs often freeze the vision encoder. Why did OpenVLA fine-tune it, and what was the effect?
8. Why does cross-entropy over bins lose information compared with L1 regression? How did OpenVLA-OFT change the output head?
9. Why did int8 quantization look worse than int4 on the real robot, and how was that confirmed?
10. Name three steps in the LIBERO fine-tuning recipe that are not about the model at all (data or evaluation conventions).

22 Sources

References

[1] M. J. Kim, K. Pertsch, S. Karamcheti, et al. *OpenVLA: An Open-Source Vision-Language-Action Model.* CoRL 2024. https://arxiv.org/abs/2406.09246

[2] S. Karamcheti, S. Nair, A. Balakrishna, P. Liang, T. Kollar, D. Sadigh. *Prismatic VLMs: Investigating the Design Space of Visually-Conditioned Language Models.* ICML 2024. https://arxiv.org/abs/2402.07865

[3] M. J. Kim, C. Finn, P. Liang. *Fine-Tuning Vision-Language-Action Models: Optimizing Speed and Success* (OpenVLA-OFT). RSS 2025. https://arxiv.org/abs/2502.19645

[4] B. Häon, K. Stocking, I. Chuang, C. Tomlin. *Mechanistic Interpretability for Steering Vision-Language-Action Models.* CoRL 2025. https://arxiv.org/abs/2509.00328

[5] OpenVLA release code and config: https://huggingface.co/openvla/openvla-7b (`modeling_prismatic.py`, `config.json`); https://github.com/openvla/openvla (`prismatic/vla/action_tokenizer.py`, LIBERO eval scripts).

Background on RT-1, RT-2, Open X-Embodiment, Octo, π_0 and FAST is summarized from general knowledge of those papers, not re-read for this guide. Llama 2 internals (layer count, widths, RMSNorm, SwiGLU, untied embeddings) come from the public Llama 2 7B configuration.